

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15

Total Marks:100

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I Jeevan Raaja S (192421005) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

<i>Criteria</i>	<i>Conceptual Understanding</i>	<i>Accuracy of Answers</i>	<i>Application of Knowledge</i>	<i>Completeness of Response</i>	<i>Clarity & Presentation</i>	<i>Total</i>
<i>Weightage</i>	30%	20%	30%	10%	10%	
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						

Signature of the student:

Total Marks Awarded

Annexure A

Short Answer Test

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence. (5 Marks)

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance. (5 Marks)

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed. (5 Marks)

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability. (5 Marks)

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. (5 Marks)

6. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy. (5 Marks)

7. A reinforcement learning agent fails to learn an optimal policy in a simulated environment. Investigate causes such as poor reward design, insufficient exploration, or incorrect state representation. Explain debugging methods and recommend improvements like reward shaping, exploration strategies, and tuning hyperparameters. (5 Marks)

8. A clustering algorithm like K-means produces inconsistent cluster assignments for similar datasets. Analyze possible reasons such as poor initialization, incorrect number of clusters, or sensitivity to outliers. Propose debugging approaches and optimization strategies like K-means++, normalization, and silhouette analysis. (5 Marks)

9. A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling. (5 Marks)

10. A natural language processing (NLP) model using recurrent neural networks (RNNs) suffers from poor long-term dependency learning. Examine causes such as vanishing gradients and limited memory capacity. Describe debugging steps and propose solutions like LSTM/GRU units, attention mechanisms, and improved training strategies. (5 Marks)

11. A support vector machine (SVM) model trained for classification is producing very low accuracy on both training and test datasets. Analyze possible causes such as improper kernel selection, incorrect hyperparameter tuning (C and gamma), or poor feature scaling. Describe systematic debugging steps to identify the issue and suggest optimization techniques like kernel tuning, normalization, and grid search to improve model performance. (5 Marks)

12. A K-Means clustering algorithm is applied to a dataset, but the clusters formed are highly imbalanced and do not reflect the actual data distribution. Investigate possible issues such as incorrect choice of K, poor initialization of centroids, or presence of outliers. Explain debugging approaches and suggest improvements like the Elbow method, K-Means++, and outlier removal techniques. (5 Marks)

13. A recurrent neural network (RNN) designed for time-series prediction shows inconsistent outputs and fails to capture long-term dependencies. Analyze possible causes such as vanishing gradients, insufficient sequence length, or improper architecture design. Describe debugging steps and propose optimization techniques such as using LSTM/GRU units, gradient clipping, and sequence normalization. (5 Marks)

14. A Naïve Bayes classifier used for text classification performs poorly when applied to real-world data containing noisy and irrelevant features. Examine possible issues such as violation of feature independence assumption, improper preprocessing, or imbalanced dataset. Explain debugging strategies and suggest improvements like feature selection, smoothing techniques, and data balancing methods. (5 Marks)

15. A reinforcement learning agent trained to solve a navigation problem fails to learn an optimal policy and shows erratic behavior. Analyze possible causes such as sparse rewards, improper exploration-exploitation balance, or incorrect reward function design. Describe debugging approaches and recommend optimization strategies like reward shaping, epsilon decay tuning, and policy optimization methods to improve learning performance. (5 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	30	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	10	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence.

Answer

A **Perceptron** is a supervised learning algorithm used for binary classification. According to the Perceptron Convergence Theorem, it should converge if the dataset is linearly separable. If it fails to converge, the problem usually lies in the implementation or parameter settings.

Possible Causes

1. Improper Learning Rate

- A learning rate that is too high causes large weight updates, making the model unstable.
- A learning rate that is too low results in very slow convergence.

2. Incorrect Weight Initialization

- Extremely large or identical initial weights can prevent efficient learning.
- Random initialization with small values is recommended.

3. Bias Handling Errors

- Missing or incorrectly updated bias values shift the decision boundary, causing incorrect classification.

4. Incorrect Update Rule

The perceptron update rule is:

$$w_{new} = w_{old} + \eta(y - \hat{y})x$$

Any mistake in this formula, such as using incorrect labels or omitting the bias update, prevents convergence.

Debugging Strategies

- Verify that the dataset is truly linearly separable.
- Print weights and bias after each iteration.
- Check whether the prediction and actual labels are correctly compared.
- Validate the implementation of the perceptron update rule.
- Plot the decision boundary after every epoch.
- Monitor the number of misclassified samples during training.

Optimization Techniques

- Use an appropriate learning rate (e.g., 0.01 or 0.1).
- Initialize weights randomly with small values.
- Normalize input features before training.
- Shuffle the training data before each epoch.
- Apply early stopping once all samples are correctly classified.

- Continuously monitor training accuracy and loss.

Conclusion

A perceptron trained on a linearly separable dataset should always converge if implemented correctly. Careful debugging of the learning rate, weight initialization, bias handling, and update rule, along with proper optimization techniques, ensures stable and faster convergence.

2. **While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance.**

Answer

A **Multilayer Perceptron (MLP)** is a feed-forward neural network trained using the **backpropagation algorithm**. During training, the model may converge slowly or become trapped in local minima due to several factors affecting the learning process.

Possible Causes

1. Vanishing Gradients

- In deep neural networks, gradients become extremely small while propagating backward.
- Earlier layers receive minimal weight updates, slowing learning.

2. Inappropriate Activation Functions

- Activation functions such as Sigmoid or Tanh may saturate, causing very small gradients.
- This reduces learning efficiency.

3. Poor Weight Initialization

- Very large or very small initial weights lead to unstable learning.
- Improper initialization may also cause exploding or vanishing gradients.

4. Suboptimal Learning Rate

- A high learning rate causes oscillations and unstable training.
- A low learning rate results in very slow convergence.

Step-by-Step Debugging Approach

1. Verify that the training data is correctly preprocessed and normalized.
2. Monitor the training and validation loss after each epoch.
3. Inspect gradient values to detect vanishing or exploding gradients.
4. Check whether the activation functions are appropriate for the network.
5. Examine the weight initialization method.
6. Evaluate learning curves to identify overfitting or underfitting.
7. Compare training and validation accuracy to assess model performance.

Optimization Techniques

- Use **ReLU** or **Leaky ReLU** activation functions instead of Sigmoid.
- Apply **Xavier** or **He initialization** for weights.
- Normalize input features using **Batch Normalization**.
- Use adaptive optimizers such as:
 - Adam
 - RMSProp
 - AdaGrad
- Apply learning rate scheduling to gradually reduce the learning rate during training.
- Use Mini-Batch Gradient Descent for faster and more stable convergence.
- Employ Early Stopping to prevent overfitting.

Advantages of Optimization

- Faster convergence.
- Stable training process.
- Better prediction accuracy.
- Reduced chances of getting trapped in local minima.
- Improved generalization on unseen data.

Conclusion

Slow convergence in an MLP is commonly caused by vanishing gradients, unsuitable activation functions, poor weight initialization, or incorrect learning rates. Systematic debugging combined with techniques such as Batch Normalization, adaptive optimizers (Adam, RMSProp), proper initialization, and ReLU activation significantly improves convergence speed, stability, and overall model performance.

3. **A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed.**

Answer

A **Genetic Algorithm (GA)** is an optimization technique inspired by natural evolution. It searches the hypothesis space by evolving a population of candidate solutions using selection, crossover, and mutation. If the algorithm produces inconsistent and suboptimal solutions, several factors may be responsible.

Possible Causes

1. Improper Chromosome Encoding

- Incorrect representation of chromosomes may not accurately model the problem.
- This leads to invalid or poor-quality solutions.

2. Incorrect Fitness Function

- A poorly designed fitness function fails to evaluate candidate solutions correctly.

- The algorithm may converge toward non-optimal solutions.

3. Premature Convergence

- The population becomes too similar early in the evolution process.
- As a result, the algorithm gets trapped in local optima.

4. Lack of Population Diversity

- Low diversity reduces exploration of the search space.
- The algorithm repeatedly generates similar solutions.

Step-by-Step Debugging Approach

1. Verify that chromosome encoding correctly represents all possible solutions.
2. Test the fitness function with known examples to ensure accurate evaluation.
3. Monitor the fitness value of each generation.
4. Check crossover and mutation operations for implementation errors.
5. Measure population diversity during evolution.
6. Compare results from multiple independent runs.
7. Analyze convergence graphs to detect premature convergence.

Optimization Techniques

- **Mutation Tuning:** Increase or adjust the mutation rate to introduce new genetic variations and maintain diversity.
- **Selection Pressure Adjustment:** Use methods such as Tournament Selection or Roulette Wheel Selection to balance exploration and exploitation.
- **Elitism:** Preserve the best-performing individuals in each generation to prevent the loss of high-quality solutions.
- Increase the population size to improve search capability.
- Use adaptive crossover and mutation probabilities.
- Apply multiple runs and select the best-performing solution.

Advantages of Optimization

- Improves solution quality.
- Prevents premature convergence.
- Maintains genetic diversity.
- Increases convergence speed.
- Produces more consistent results across multiple runs.

Conclusion

Suboptimal performance in a Genetic Algorithm is mainly caused by improper chromosome encoding, poor fitness function design, premature convergence, and insufficient population diversity. By systematically debugging these issues and applying optimization techniques such as mutation tuning, balanced selection pressure, elitism, and adaptive genetic operators, the algorithm can achieve faster convergence, better exploration of the hypothesis space, and more accurate solutions.

